Introducción a la concurrencia

Sistemas Operativos I

Oliver Díaz

Grau d'Enginyeria Informàtica

¿Qué hacen estas personas?

La concurrencia también se aplican a las máquinas

```
  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 2407 oslab     20   0 3185276 340876 180664 S 4.651 16.83   0:12.36 /usr/lib64/firefox/firefox
 2473 oslab     20   0 3106120 508488 293368 S 1.993 25.11   0:04.85 /usr/lib64/firefox/firefox -contentproc +
 2565 oslab     20   0 3075624 472424 257440 S 1.661 23.33   0:04.14 /usr/lib64/firefox/firefox -contentproc +
   33 root      20   0       0      0      0 I 0.664 0.000   0:00.59 [kworker/1:1-events]
   10 root      20   0       0      0      0 S 0.332 0.000   0:00.09 [ksoftirqd/0]
  269 root      20   0       0      0      0 I 0.332 0.000   0:00.82 [kworker/0:3-events_power_efficient]
 1438 root      20   0 1070516 143096  67200 S 0.332 7.066   0:04.48 /usr/bin/X :0 -seat seat0 -auth /run/lig+
 1660 oslab     20   0  172580   1492   1492 S 0.332 0.074   0:00.71 /usr/bin/VBoxClient --draganddrop
 2724 oslab     20   0   43572   4380   3760 R 0.332 0.216   0:00.07 top -c
    1 root      20   0  172284  11544  10108 S 0.000 0.570   0:02.05 /usr/lib/systemd/systemd --switched-root+
```

Definición

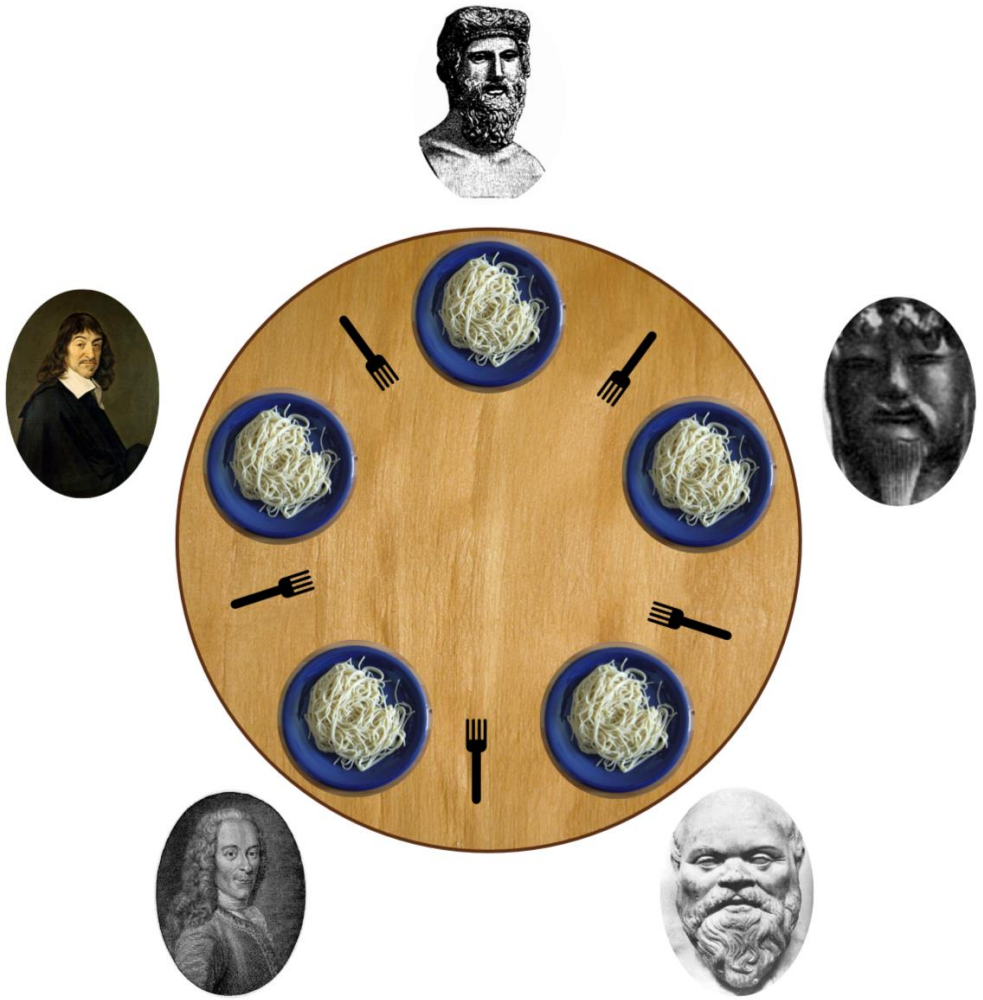
- "[…] concurrencia se refiere a la habilidad de distintas partes de un programa, algoritmo, o problema de **ser ejecutado en desorden o en orden parcial, sin afectar el resultado final**. Los cálculos (operaciones) pueden ser ejecutados en **múltiples procesadores, o ejecutados en procesadores separados físicamente o virtualmente en distintos hilos de ejecución**."

Wikipedia

Definición

- "[…] habilidad de un programa para realizar **varias tareas al mismo tiempo**, de forma independiente y simultánea. En un sistema operativo, puede haber **múltiples procesos o hilos de ejecución que comparten los mismos recursos** del sistema, como la memoria, el CPU y los dispositivos de entrada/salida.

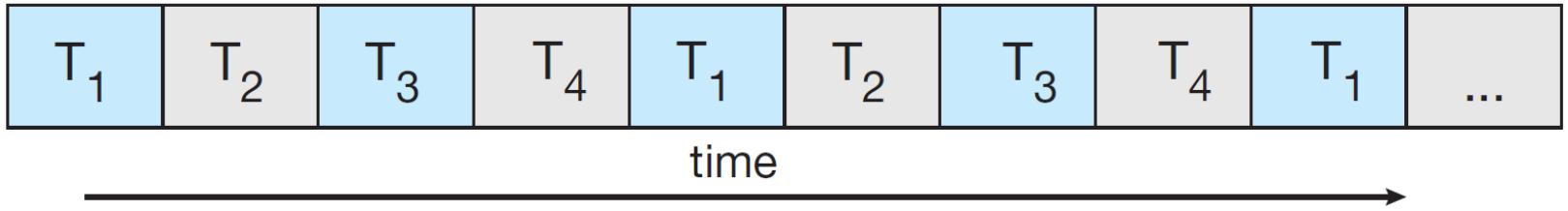
ChatGPT



La cena de los fílósofos

Introducción

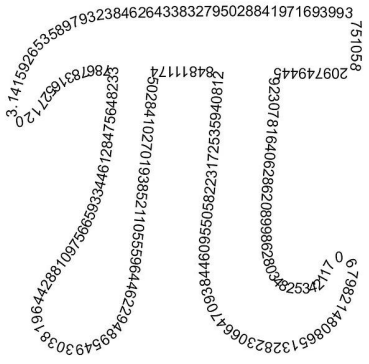
- Como programadores y programadoras, estamos acostumbrados a pensar de forma **secuencial**.
  - Se ejecuta la función principal (*main* en C).
  - Se ejecutan las instrucciones del programa de manera secuencial.

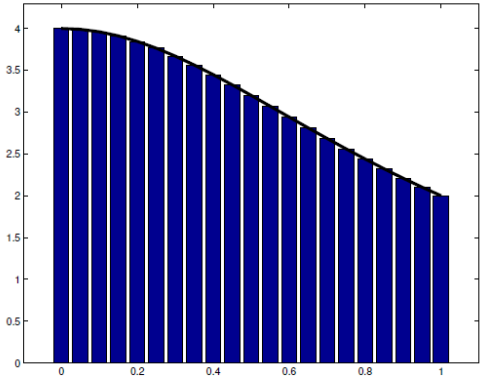


- La **concurrencia** es una forma de estucturar una solución que puede ser parallelizable (¡aunque no siempre es posible!).

¿Se pueden ejecutar partes del códigos al mismo tiempo?

- `fork()` -> Procesos independientes se comunican entre sí para reparetirse el trabajo.

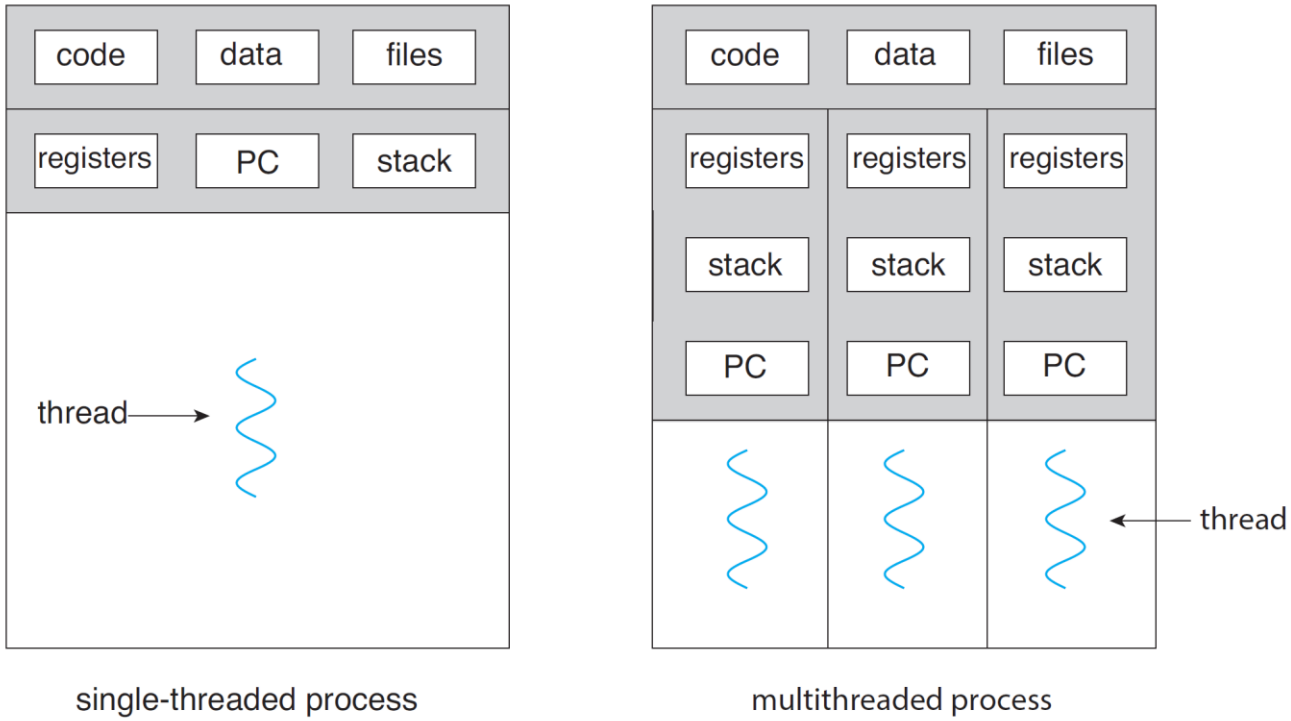




- Los SOs actuales nos ofrecen herramientas para que un determinado proceso puede ejecutar diversas partes del código al mismo tiempo -> concepto de **hilo**
- El SO nos permite gestionar la concurrencia a través de multiples procesos o multiples hilos.

Hilo

- **Hilo (thread)**: Unidad básica de ejecución dentro de un proceso
  - ID de hilo, contador de programa (PC), un conjunto de registros y una pila.
  - Los hilos del mismo proceso comparten su sección de código, datos y otros recursos del SO (ficheros abiertos y señales)
  - Un proceso tradicional tiene un único hilo de control. Si un proceso tiene varios hilos de control, puede realizar más de una tarea a la vez.



Silberschatz, A., Peterson, J. L., & Galvin, P. B. (2018). *Operating system concepts*. Wiley

¿Por qué los hilos?

- Hasta ahora hemos considerado que un proceso se ejecuta en un único hilo (thread).

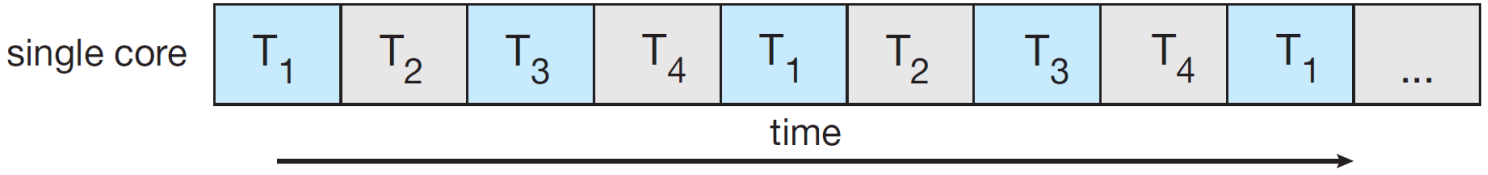
- SOs modernos permiten a los procesos ser ejecutados utilizando multiples hilos (**multithreading**).

- La programación multihilo es muy popular hoy en dia donde nos encontramos con **sistemas multicore** que nos ofrencen las CPUs.

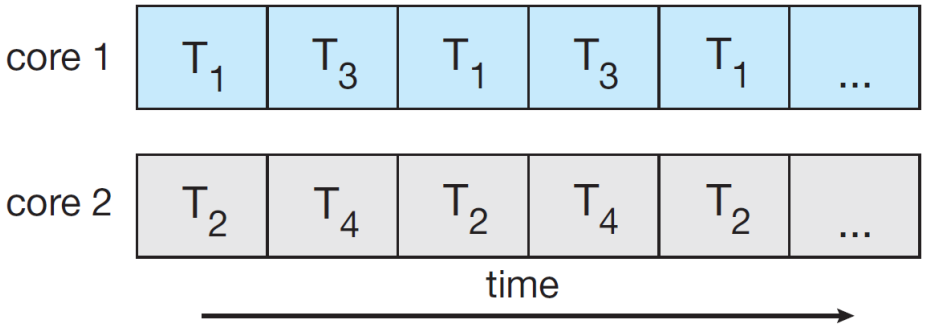
Multicore

- **Multicore**: sistemas con múltiples núcleos de computación en un solo chip de procesamiento, dónde cada núcleo aparece como una CPU separada para el SO.

Ejecución concurrente en un sistema de un solo núcleo.



Ejecución en paralelo en un sistema multinúcleo.



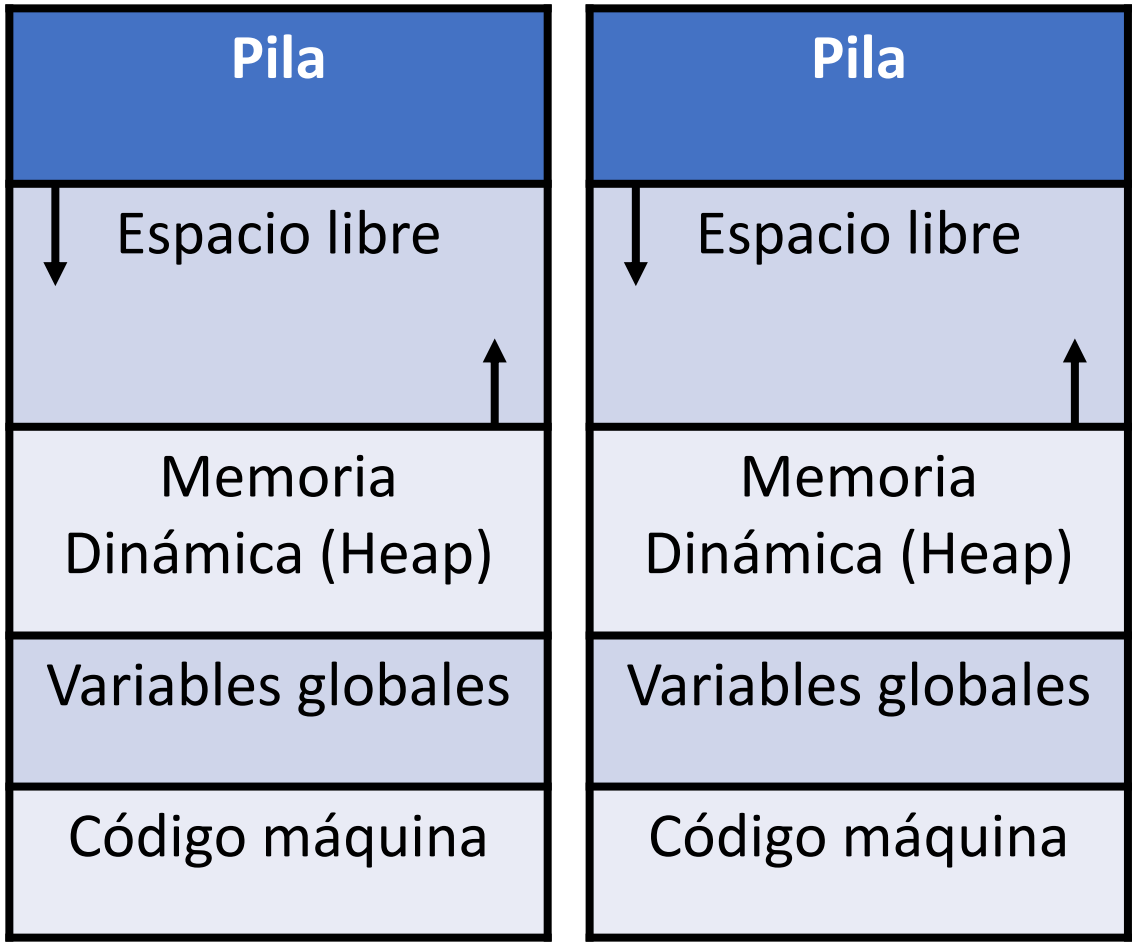
Silberschatz, A., Peterson, J. L., & Galvin, P. B. (2018). *Operating system concepts*. Wiley

Ejemplo procesos multihilos

- **Navegador internet**
  - Hilo 1: muestra imágenes/texto
  - Hilo 2: recoge datos de la red
- **Navegador de ficheros**
  - Cada hilo puede generar el thumbnail de cada imagen por separado
- **Procesador de textos**
  - Hilo 1: Muestra gráficos
  - Hilo 2: Leer información de teclado
  - Hilo 3: Revisión ortográfica y gramatical
- …

Concurrencia con procesos

- Cada proceso tiene un **espacio de memoria independiente**. Por defecto, no pueden escribir en el espacio de memoria de otro proceso.
- Para comunicarse entre sí hace falta utilizar los servicios del SO (fichero compartido, tuberia, red,…)

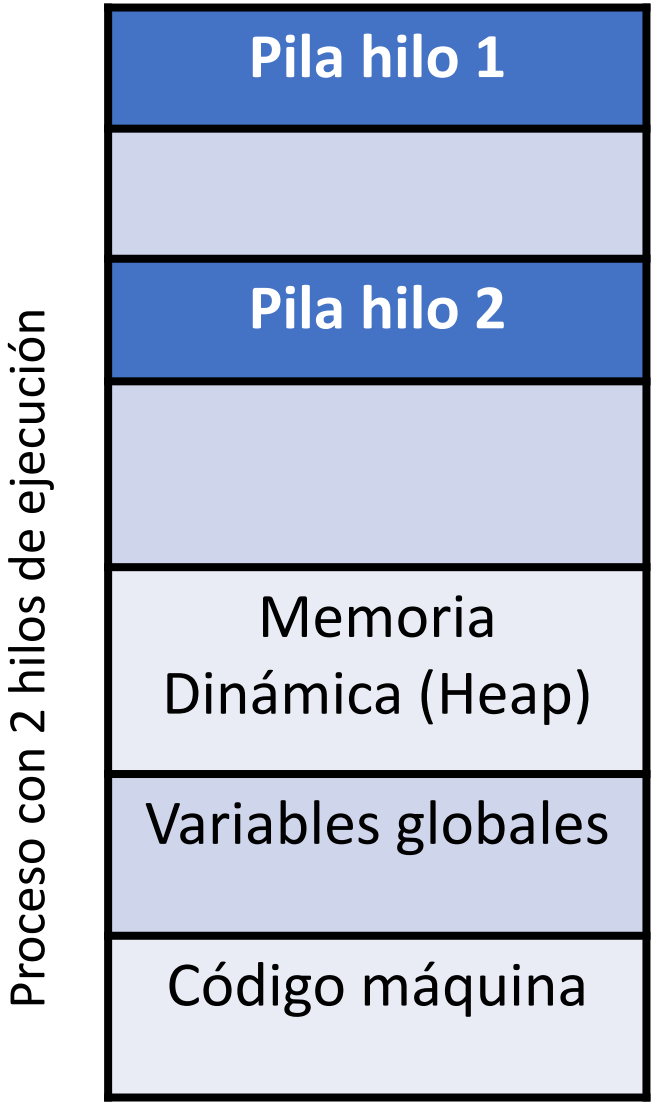


Proceso 1
con 1 hilo de ejecución

Proceso 2
con 1 hilo de ejecución

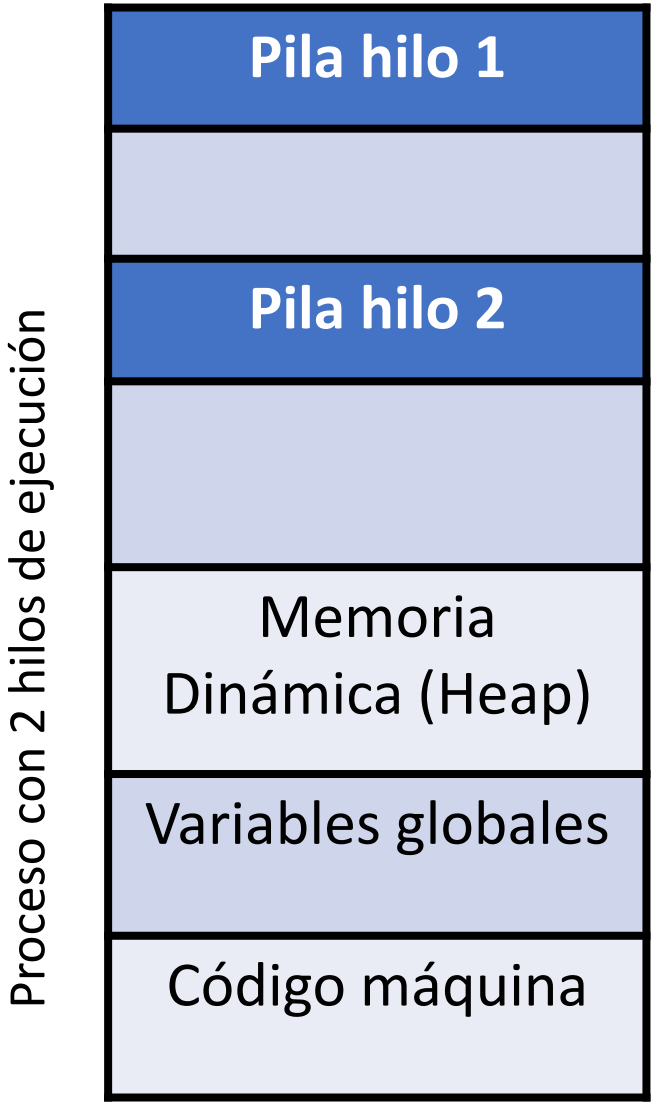
Concurrencia con hilos

- Los hilos de un proceso **comparten el espacio de memoria del proceso**. Cada hilo tiene su propia pila y registros de la CPU y puede ejecutar una parte diferente del código.
- Para comunicarse entre si pueden utilizar el espacio de memoria que comparten.



Concurrencia con hilos

```
int global;

void funcio()
{
  int a;

  // Aixo ho executen els dos fils
}

void main(...)
{
   int fil;

   // Aixo ho executa un sol fil, una sola pila

   fil = crear_fil_nou(&funcio); // Retorna id fil creat

   // Aixo ho executa un sol fil, hi ha dues piles

   funcio();

   esperar_que_acabi_fil(fil);
}
```



Concurrencia con hilos

- **Elementos de un proceso**
  - Espacio de direcciones (código, variables globales, mem. dinámica, …)
  - Ficheros abiertos
  - Lista de hilos del proceso
  - Otros
- **Elementos de un hilo**
  - Contador de programa (Program Counter; PC)
  - Registros de la CPU
  - Pila
  - Estado (preparado, bloqueado, ejecución)

Concurrencia con hilos

- En la actualidad, el SO realiza la planificación con los hilos.
- Cada hilo tiene un estado diferente y **el planificador realiza el cambio de contexto a nivel de hilo.**
  - En el tema de planificación hablábamos de “tareas” en vez de “procesos”. En el fondo se planifican “hilos”.
- Cada hilo puede estar ejecutándo una parte diferente del código del proceso. Incluso el mismo codigo puede ser ejecutado por diferentes hilos al mismo tiempo.

Beneficios programación multihilos

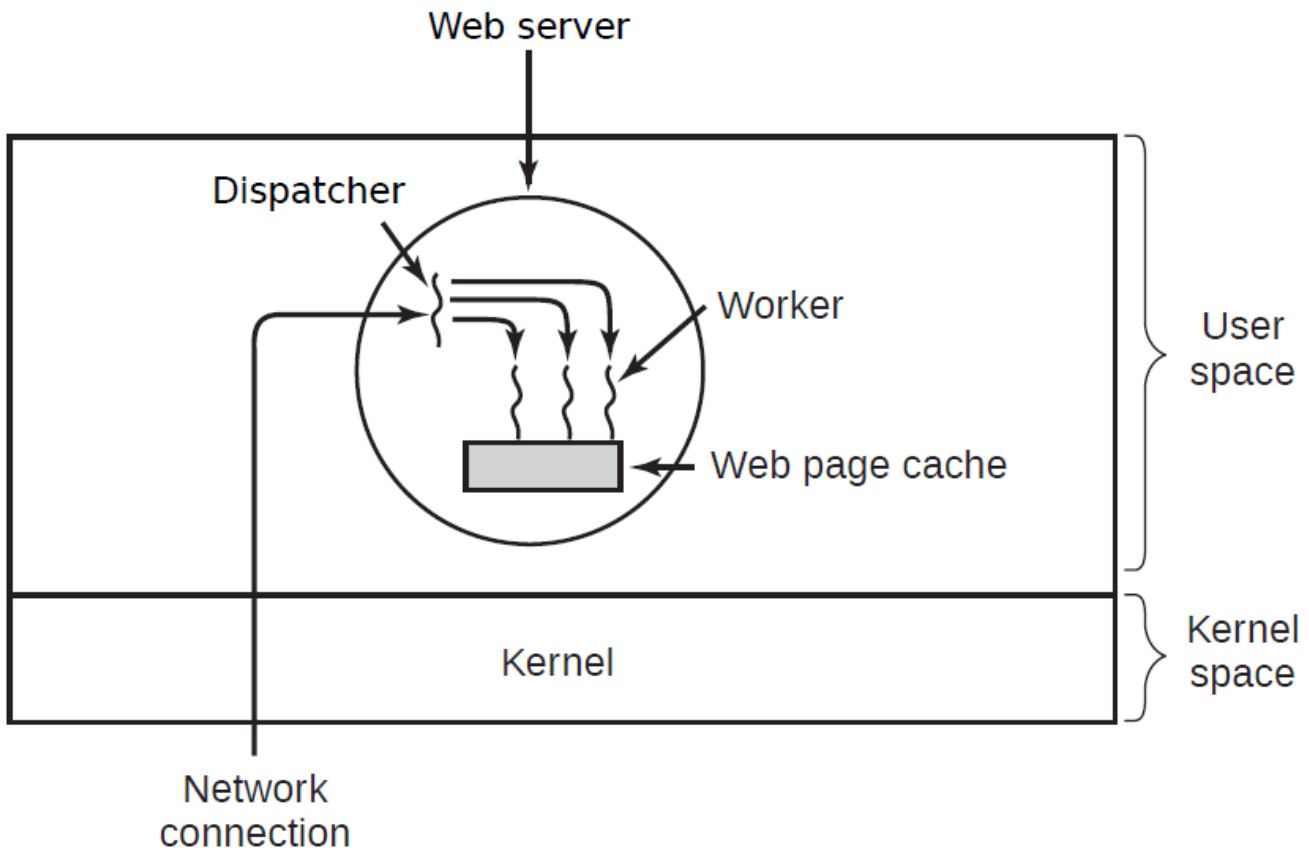
- Tipicamente **crear un hilo es menos costoso** (tiempo, memoria) que un hijo.
- Mejora el tiempo de respuesta de las aplicaciones. **Cambio de contextos entre hilos es normalmente más rápido** que entre procesos.
- **Mejor gestión de recursos**. Se comparten códigos y datos, utilizando el mismo espacio de direcciones (más facil que comunicación de procesos)
- **Simplificación del código** para tratar eventos asíncronos (raton, teclado, red)
- Grandes beneficios sobre todo en arquitecturas multiprocesos, donde hilos pueden ejecutarse en paralelos en diferentes cores.
- También tienen beneficios en arquitecturas uniprocesador.

Procesos vs hilos

- **En los procesos**
  - La comunicación puede ser entre procesos ejecutandose en el mismo o diferente ordenador
  - La comunicación se realiza mediante servicios que ofrece el SO
- **En los hilos**
  - Todos los hilos de un proceso se ejecutan en el mismo ordenador.
  - La comunicación se realiza a través del espacio de memoria del proceso (p. ej. Variables globales). No es necesario utilizar los servicios del SO.
  - La complejidad se centra en la sincronización y coordinación de los hilos porque se comunican correctamente entre si (Sistemas Operativos 2)

¿Procesos o hilos? ¿Qué opción es mejor?

Queremos diseñar un aplicación, por ejemplo un servidor web o un navegador web. ¿Que utilizaríamos? ¿multiples procesos o multiples hilos?



¿Procesos o hilos? ¿Qué opción es mejor?

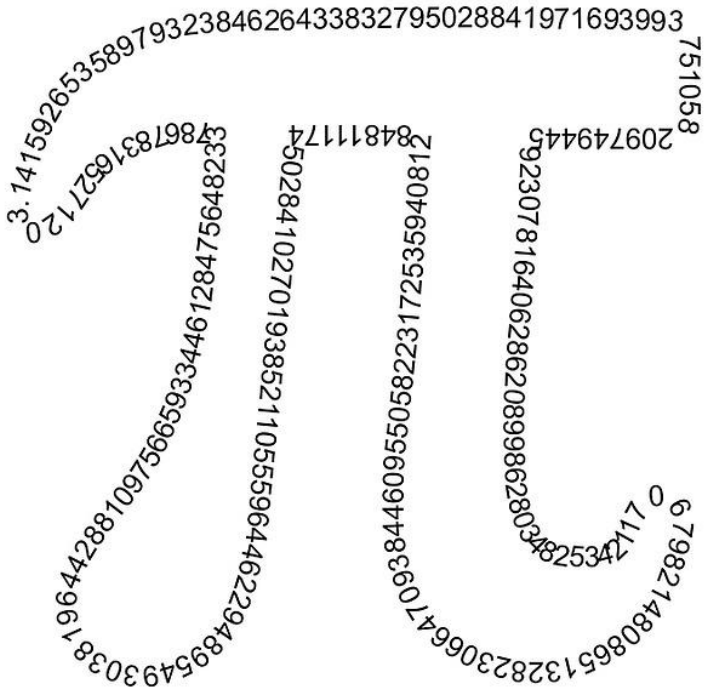
- Si la decision depende de la cantidad de **datos a compartir**.
  - Utilizar hilos si hay muchos datos a compartir.
  - Utilizar procesos si hay pocos datos a compartir.
- Si depende de la **seguridad** a implementar.
  - Si un hilo realiza una operación inválida, el SO mata todo el proceso (con todos sus hilos)
  - Si un proceso realiza una operación inválida en memoria, el SO mata el proceso que ha realizado la operación inválida, pero no el resto.

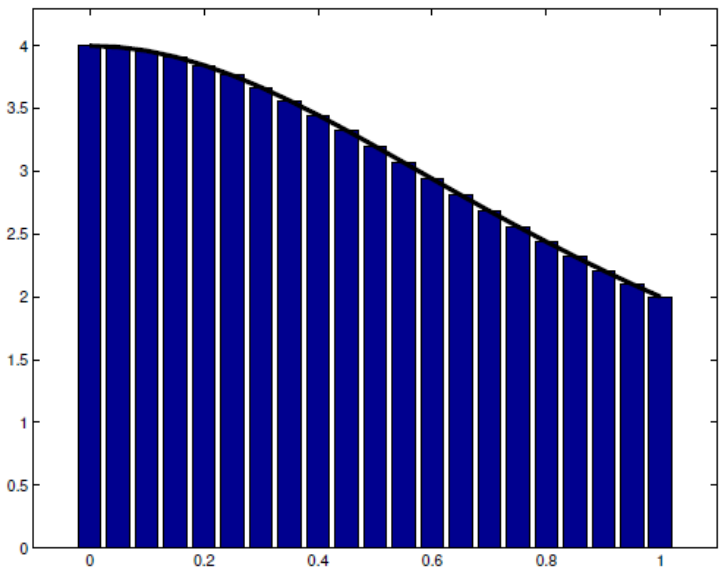
¿Procesos o hilos? ¿Qué opción es mejor?

- **Google Chrome/Firefox**: Cada tabulador es un proceso.
  - Separación de procesos o sandboxing
- **Edge**: Cada tabulador es un hilo.
  - Separación por hilos

¿Procesos o hilos? : ejercicio

$$\pi = \int_0^1 \frac{4}{1+x^2}\,dx$$



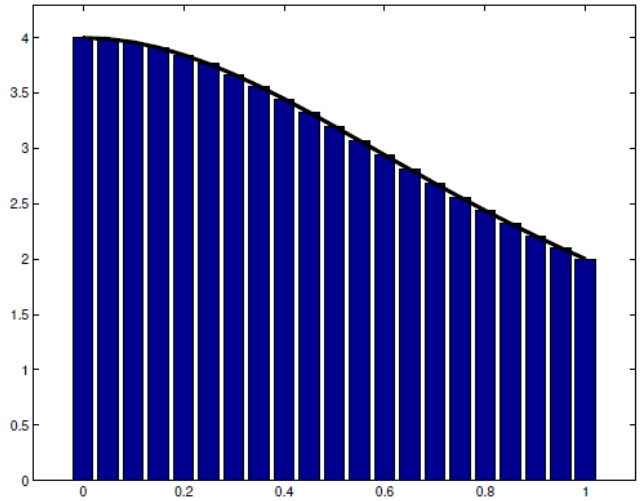



```
#define NUM_RECTS 1E09
int main()
{
  int i;
  double mid, height, width, sum = 0.0;
  double area;

  width = 1.0 / (double) NUM_RECTS;
  for(i = 0; i < NUM_RECTS; i++) {
    mid = (i + 0.5) * width;
    height = 4.0 / (1.0 + mid * mid);
    sum += height;
  }
  area  = width * sum;
  printf("pi = %e\n", area);

  return 0;
}
```

- El cálculo del área de cada rectángulo es independiente del resto de rectángulos.

¿Procesos o hilos?: ejercicio

- Implementar el código (calcul_pi_1proces.c) utilizando un solo proceso con un hilo.
  - `time calcul_pi_1proces`
- Implementar el código utilizando dos procesos, cada proceso con un hilo. (calcul_pi_2processos.c)
  - `time calcul_pi_2processos`
- Implementar el código utilizando un proceso con tres hilos: un hilo principal y dos hilos que realizan los cálculos (calcul_pi_fils.c)
  - `gcc calcul_pi_fils.c -o codigo -lpthread`
  - `time calcul_pi_fils`



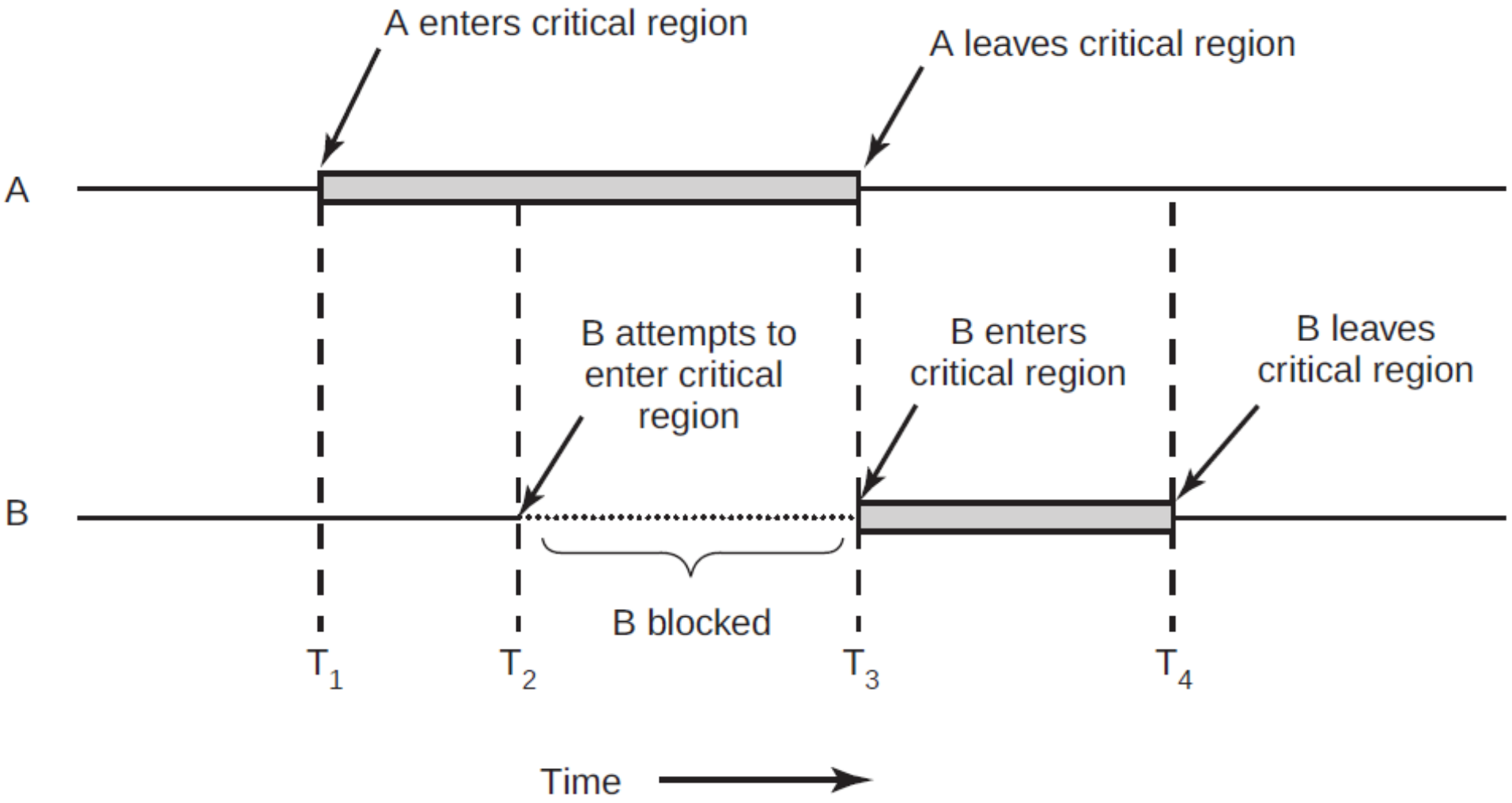
¿Procesos o hilos?: ejercicio

- El ejemplo con multiples procesos utiliza los mecanismos de comunicación que nos ofrecen los SO (tuberias)
- En el ejemplo con multiples hilos la comunicación se realiza a través de la variable global (`results`). Los dos hilos que realizan los cálculos comunican el resultados parciales al hilo principal a través de esta variable global.

¿Procesos o hilos?

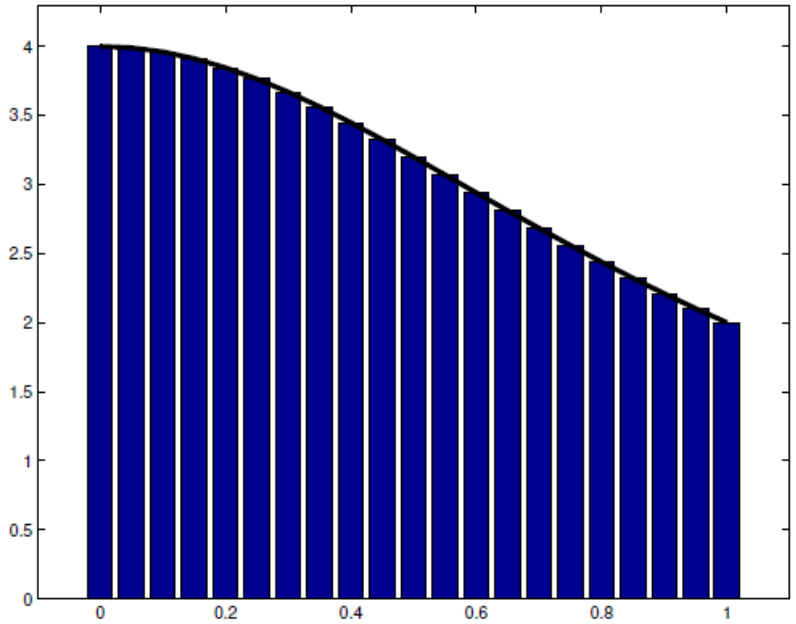
- Es necesario sincronizar los hilos/procesos entre sí cuando se accede a **recursos (variables) compartidas**.
- **Sección crítica**: parte del código de un programa que accede a un recurso compartido, como una variable o un archivo, que puede ser modificado por otros procesos o hilos concurrentes.
- **Exclusión mutua**: hilos y/o procesos diferentes no pueden acceder a la misma sección crítica en el mismo instante de tiempo.

Exclusión mutua



Exclusión mutua

- Ver ejemplo calcul_pi_fils_mutex.c
- Funciones `lock` y `unlock`.



Conclusiones

- Un proceso es una instancia de un programa mapeado a memoria con toda la información asociada a el (espacio de direcciones, ficheros abiertos, …)
- Un hilo es la unidad que ejecuta un proceso. Cada hilo tiene información asociada a él (estado, registros CPU, etc.)
- Un proceso puede tener multiples hilos de ejecución, y éstos pueden comunicarse (compartir datos) entre sí de forma sencilla ya que comparten el mismo espacio de direcciones.

Conclusiones

- Un **sistema concurrente** admite más de una tarea al permitir que todas las tareas progresen. Por el contrario, un **sistema paralelo** puede realizar más de una tarea simultáneamente.
- Por lo tanto, es posible tener concurrencia sin paralelismo.
- Antes de la llegada de las arquitecturas multiprocesador y multinúcleo, la mayoría de los sistemas informáticos tenían un solo procesador. Los programas de planificación de CPU se diseñaron para proporcionar la ilusión de paralelismo al cambiar rápidamente entre procesos, lo que permite que cada proceso progrese. Dichos procesos se estaban ejecutando al mismo tiempo, pero no en paralelo.
- No asociar programación multihilos con sistemas multiprocesadores.
- Las ventajas de la programación multihilos son evidentes también con sistemas uniprocesadores.

Bonus: ¿Cómo ver los hilos?

- pstree

khushi@ubuntulinux: ~/gfg

```
khushi@ubuntulinux:~/gfg$ pstree
systemd─┬─ModemManager───2*[{ModemManager}]
        ├─NetworkManager───2*[{NetworkManager}]
        ├─accounts-daemon───2*[{accounts-daemon}]
        ├─acpid
        ├─apache2───5*[apache2]
        ├─avahi-daemon───avahi-daemon
        ├─bluetoothd
        ├─colord───2*[{colord}]
        ├─cron
        ├─cups-browsed───2*[{cups-browsed}]
        ├─cupsd
        ├─dbus-daemon
        ├─fwupd───4*[{fwupd}]
        ├─gdm3─┬─gdm-session-wor─┬─gdm-x-session─┬─Xorg───{Xorg}
        │      │                 │               ├─gnome-session-b───3*[{gnome-+
        │      │                 │               └─2*[{gdm-x-session}]
        │      │                 └─2*[{gdm-session-wor}]
        │      ├─gdm-session-wor─┬─gdm-x-session─┬─Xorg───{Xorg}
        │      │                 │               ├─gnome-session-b─┬─ssh-agent
        │      │                 │               │                 └─2*[{gnome-+
        │      │                 │               └─2*[{gdm-x-session}]
        │      │                 └─2*[{gdm-session-wor}]
        │      └─2*[{gdm3}]
        ├─gnome-keyring-d───3*[{gnome-keyring-d}]
        ├─ibus-daemon─┬─ibus-dconf───3*[{ibus-dconf}]
        │             ├─ibus-engine-sim───2*[{ibus-engine-sim}]
        │             ├─ibus-extension-───3*[{ibus-extension-}]
        │             ├─ibus-ui-gtk3───3*[{ibus-ui-gtk3}]
        │             └─2*[{ibus-daemon}]
        ├─ibus-x11───2*[{ibus-x11}]
        ├─irqbalance───{irqbalance}
        ├─2*[kerneloops]
        ├─networkd-dispat
        ├─nvidia-persiste
        ├─polkitd───2*[{polkitd}]
        ├─rsyslogd───3*[{rsyslogd}]
        ├─rtkit-daemon───2*[{rtkit-daemon}]
        ├─snapd───22*[{snapd}]
        ├─switcheroo-cont───2*[{switcheroo-cont}]
        ├─systemd─┬─(sd-pam)
        │         ├─at-spi-bus-laun─┬─dbus-daemon
        │         │                 └─3*[{at-spi-bus-laun}]
        │         ├─at-spi2-registr───2*[{at-spi2-registr}]
        │         ├─dbus-daemon
        │         ├─dconf-service───2*[{dconf-service}]
        │         ├─gjs───10*[{gjs}]
        │         ├─gnome-keyring-d───3*[{gnome-keyring-d}]
        │         ├─gnome-session-b───3*[{gnome-session-b}]
        │         ├─gnome-session-c───{gnome-session-c}
        │         ├─gnome-shell─┬─ibus-daemon─┬─ibus-dconf───3*[{ibus-dconf}]
        │         │              │              ├─ibus-engine-sim───2*[{ibus-engi+
        │         │              │              └─2*[{ibus-daemon}]
        │         │              └─11*[{gnome-shell}]
        │         ├─goa-daemon───3*[{goa-daemon}]
        │         ├─goa-identity-se───2*[{goa-identity-se}]
```

Gracias